Introduction to Computing (CS 101)

Introduction to Functions

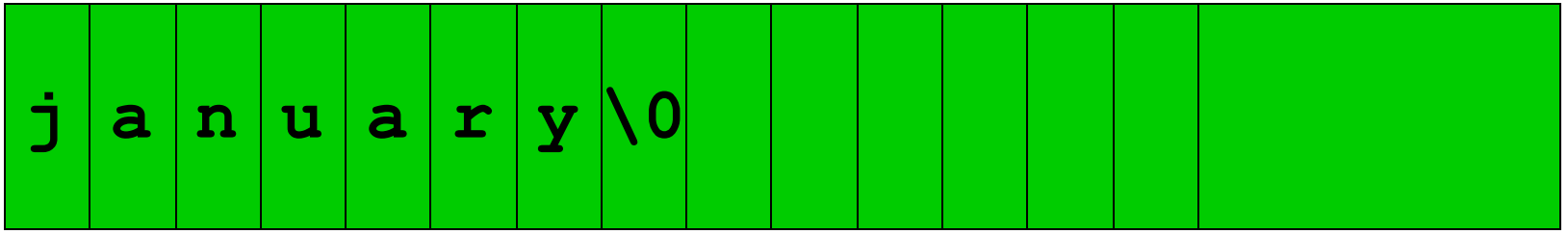
T. Venkatesh & John Jose

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Strings : Arrays of Characters

- How to store ? “I study in IITG”
- String enclosed in double quotes
- Stored as ASCII codes of each character
- Ended with the ASCII code 0: null: '\0‘
- Strings is a one dimensional array of characters
- `char month1[ ] = {'j','a','n','u','a','r','y',’\0’};`
- `char month1[ ] = “january”;`



C terminator: ’\0’

String Operations

- **`string.h`**
- **`strcat()`:** concatenate two strings. Answer in the first string.
- **`strcmp()`:** compare two strings using ASCII code
  - < 0 : if first string < second
  - = 0 : if first string = second
  - > 0 : if first string > second
- **`strcpy()`:** copies second string on the first including null character
- **`strlen()`:** Number of characters in the string, excluding null character

String Operations

```
#include <stdio.h>
#include <string.h>

void main()
{
  char str1[30], str2[30];
  int cmpResult;
  printf("\nEnter the first string: ");
  scanf("%s", str1);
  printf("\nEnter the second string: ");
  scanf("%s", str2);

  cmpResult = strcmp(str1, str2);
```

String Operations

```
 if (cmpResult < 0)
    printf("str1 < str2 ");
 else if (cmpResult == 0)
    printf("str1 = str2 ");
  else
    printf("str1 > str2 ");

  printf("\nNow we will overwrite str2 with
     str1");
  strcpy(str2, str1);

  printf("\nstr1 = %s\n", str1);
  printf("\nstr2 = %s\n", str2);
}
```

Sample Programs For Practice

- Search for an element in an array
- Check for occurrences of an element in an array
- Check the number with largest occurrences in an array
- Sort the elements in an array in increasing order
- Check whether a substring is present in a string or not
- Search the count of each vowel in a string

Functions

- Modularize a program
- They are separate modules of code
- All variables declared inside functions are local variables : Known only in function defined
- Parameters: Communicate information between functions
- Function Benefits
  - Divide and conquer : Manageable program development
  - Software reusability : Use existing functions as building blocks for new programs and
  - Abstraction : hide internal details
  - Avoids code repetition

Functions

- Analogy - Boss asks worker to complete task
  - Worker gets information, does task, returns result
  - Information hiding: boss does not know details
- Invoking functions
  - Provide function name and arguments (data)
  - Function performs operations or manipulations
  - Function returns results

Function Definitions

- Function definition format

```
return-value-type function-name (parameter-list) {
    declarations and statements
}
```

- Function-name: any valid identifier
- Return-value-type: data type of the result (default **int**)
  - **void** - function returns nothing
- Parameter-list: comma separated list, declares parameters (default **int**)

```
int average (A, n){
// Function body
//Return result
}
```

Function Definitions

- Function definition format

```
return-value-type function-name (parameter-list) {
    declarations and statements
}
```

- Declarations and statements: function body (block)
  - Variables can be declared inside blocks (can be nested)
  - A function can not be defined inside another function

- Returning control
  - If nothing returned : **return;**
  - If something returned : return *expression*;

Functions

- A C program is made up of one or more functions, one of which is main( ).
- Execution always begins with main( )
  - No matter where it is placed in the program.
- When program control encounters a function name, the function is **called** (**invoked**).
  1. Program control passes to the function.
  2. The function is executed.
  3. Control is passed back to the calling function.

Sample Function Call

```
#include <stdio.h>
 int main ( ) {

  printf("Hello World!\n");
  return 0 ;
}
```

printf is the name of a **predefined function** in the stdio library

this statement is known as a **function call**

A string we are **passing** as an **argument** (**parameter**) to the printf function

Sample -Defined Function

```
#include <stdio.h>
void PrintMessage(void);
int main(){
   PrintMessage();
   return 0 ;
}
void PrintMessage(void){
  printf("A MSG :\n\n");
  printf("Nice day!\n");
}
```

Function Prototype/ Declaration

Function Call

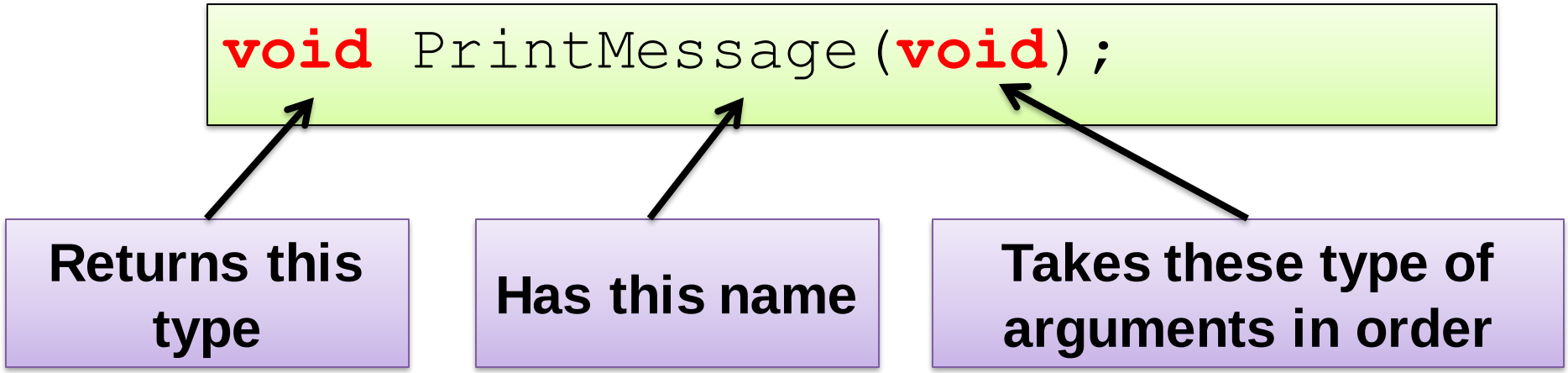
Function Header

Function Body or Definition

The Function Prototype

- Informs the compiler that there will be a function defined later

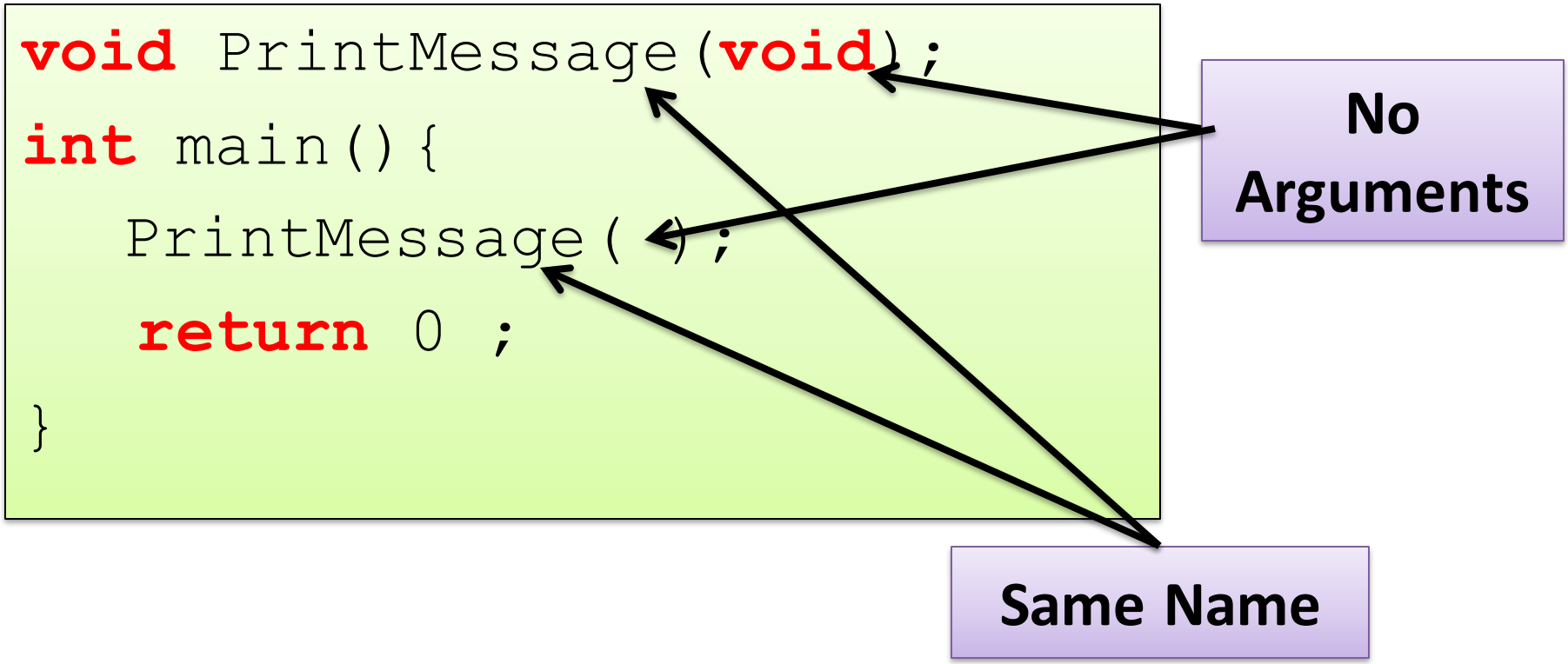
- This is needed because the function call is made before the definition -- the compiler uses it to see if the call is made properly



The Function Call

- Passes program control to the function
- Must match the prototype in name, number of arguments, and types of arguments

```
void PrintMessage(void);
int main(){
   PrintMessage( );
   return 0 ;
}
```



The Function Definition

- Control is passed to the function by function call
  - The statements within the function body will then be executed

```
void PrintMessage(void){
  printf("A MSG :\n\n");
  printf("Nice day!\n");
}
```

- After statements in the function have completed
  - Control is passed back to the **calling function**
- In this case main( )
  - Note that the calling function does not have to be main( )

General Function Definition Syntax

type functionName (*parameter*$_1$, . . . , *parameter*$_n$) {
    variable declaration(s)
    statement(s)
}

- If there are no parameters
  - either **`functionName()`** OR **`functionName(void)`**
- There may be no variable declarations.
- If the **function type** (**return type**) is void, a return statement is not required
  - Permitted: **`return`**; OR **`return()`**;

Input Parameters to Function

```
void PrintMessage(int counter) ;

int main ( ){
    int num=10;
    PrintMessage(num);
    return 0 ;
}

void PrintMessage(int counter) {
    int i ;
    for (i=0;i<counter; i++)
      printf ("Nice day!\n") ;
}
```

matches the one **formal parameter** of type **int**

one argument of type **int**

Functions Can Return Values

```
#include <stdio.h>
float AverageTwo(int num1,int num2);
int main(){
  float ave ;
  int value1 = 5, value2 = 8 ;
  ave=AverageTwo(value1, value2) ;
  printf("The average of %d & %d
      is %f\n",value1,value2,ave);
 return 0 ;
}
float AverageTwo (int num1, int num2) {
  float a = (float)((num1+num2)/2.0);
  return    (a);
}
```

Functions Can Return Values

- Temp Convert Function in C

```
double CtoF ( double paramCel){
    return paramCel*1.8+32.0;
}
```

- This function takes an input parameter
  - Called paramCel (temp in degree Celsius)
- Returns a value
  - that corresponds to the temp in degree Fahrenheit

How to use a function?

```
#include <stdio.h>
double CtoF(double);
/*  Purpose: to convert temperature
* from Celsius to Fahrenheit *****/
int main() {
  double c, f;
  printf("Enter the degree (in Celsius): ");
  scanf("%lf", &c);
  f = CtoF(c);
  printf("Temperature (in Fahrenheit)
                              is %lf\n", f);
}
double CtoF ( double paramCel) {
  return paramCel * 1.8 + 32.0;
}
```

Function Declaration

Function Call

Function Definition

How to write function in modular way?

Declarations

```
#include <stdio.h>

double GetTemp();
double CelsToFahr(double);
void DispRes(double,double);

int main(){
  double TempC, TempF;

  TempC=GetTemp();
  TempF=CelsToFahr(TempC);
  DispRes(TempC,TempF);

  return 0;
}
```

Function Calls

```
double CelsToFahr(double Tem){
  return (Tem * 1.8 + 32.0);
}
```

```
double GetTemp (){
  double Temp;
  printf("Please enter temp in
       degrees Celsius:");
  scanf("%lf", &Temp);
  return Temp;
}
```

```
void DispRes(double CTemp,
          double Ftemp){
  printf("Original: %5.2f
          C\n", CTemp);
  printf("Equivalent: %5.2f
          F\n", FTemp);
}
```

Abstraction using Functions

- We hide details on how something is done in the function implementation, Functions abstract the implementation.
- For e.g., we don't see printf code but still use it.

```c
#include <stdio.h>

int main(){
   double TempC, TempF;

   TempC=GetTemp();
   TempF=CelsToFahr(TempC);
   DispRes(TempC,TempF);

   return 0;
}
```

```c
double CelsToFahr(double Tem){
   return (Tem * 1.8 + 32.0);
}
```

```c
double GetTemp (){
   double Temp;
   printf("Please enter temp in
          degrees Celsius:");
   scanf("%lf", &Temp);
   return Temp;
}
```

```c
void DispRes(double CTemp,
             double Ftemp){
   printf("Original: %5.2f
          C\n", CTemp);
   printf("Equivalent: %5.2f
          F\n", FTemp);
}
```

Parameter Passing

- **Actual parameters** are the parameters that appear in the function call

```
ave =AverageTwo(value1,value2) ;
```

- **Formal parameters** are the parameters that appear in the function header

```
float AverageTwo(int num1,int num2)
```

- Corresponding actual and formal parameters must be of the same data type.
- They need not have same name, but they may.
- Actual and formal parameters are matched by position.
- Each formal parameter receives the value of its corresponding actual parameter.

Parameter Passing

- Two mechanism to pass arguments to a function:
  - Pass by value
  - Pass by reference

PASS BY VALUE	PASS BY REFERENCE
Mechanism of copying the function parameter value to another variable	Mechanism of passing the actual parameters to the function
Changes made inside the function are not reflected in the original value	Changes made inside the function are reflected in the original value
Makes a copy of the actual parameter	Address of the actual parameter passes to the function
Function gets a copy of the actual content	Function accesses the original variable's content
Requires more memory	Requires less memory
Requires more time as it involves copying values	Requires a less amount of time as there is no copying

Passing by value

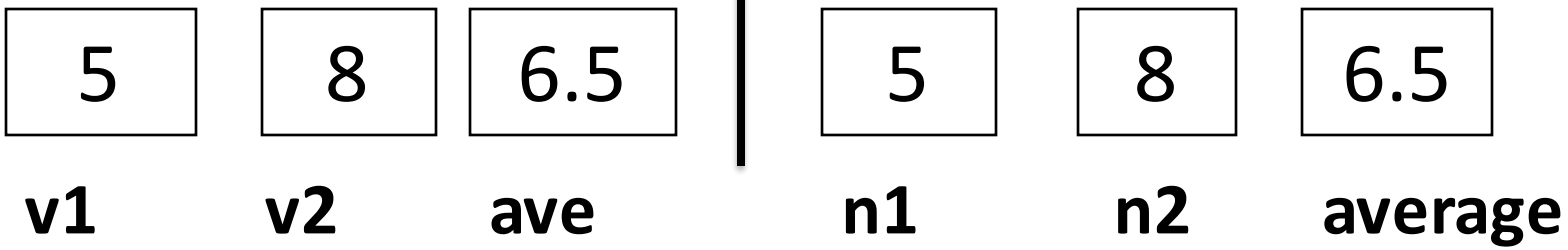
- Values of arguments are sent to called function.
- Contents of arguments in calling function are not changed, even if they are changed in the called function.
- The content of the variable is copied to the formal parameter of the function definition, thus preserving the contents of the argument in the calling function.

Passing by value

```
int main(){
   float ave ;
   int v1=5, v2=8 ;
   ave=AvgOfTwo(v1, v2);
   printf ("The average
        is %f\n", ave);
   return 0 ;
}
```

```
float AvgOfTwo(int n1,
               int n2){
   float average;
    average=(n1+n2)/2;
   return average;
}
```

Local copy of variables



Parameter Passing and Local Variables

What is the output printed?

```
int main(){
   int n1=5;
   AddOne(n1);
   printf ("In main
       n1 is %d\n",n1);
   return 0 ;
}
```

```
void AddOne (int n1){
   n1=n1+1;
   printf ("In AddOne
       n1 is %d\n",n1);
   return;
}
```

5

n1

6

n1

Local copy of variables

Changes to Local Variables Do NOT Change Other Variables in other function with the Same Name

OUTPUT

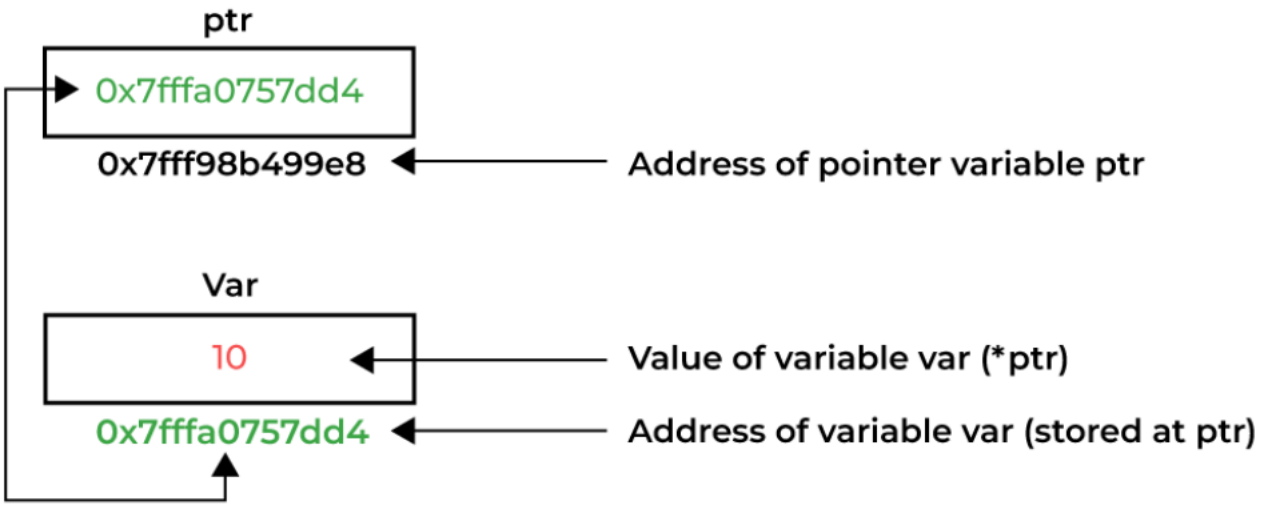
In AddOne n1 = 6

In main n1 = 5

Solution : Passing by reference

- A pointer can store the memory address of a variable.
- We can access and manipulate the data stored in that memory location using pointers.
- Ex: **`int *p`** // p stores the address of an integer variable

```
int var = 10;
int * ptr;
ptr = &var;
```



Passing by reference

- Address of Values of arguments are sent to called function.
- Change in the called function may change the contents of arguments in calling function.
- The actual parameter in the calling function and formal parameter in the called function are same.

Passing by reference

```
int main(){
   int n1=5;
   int *Pn1;
   Pn1=&n1;
   AddOne(Pn1);
   printf ("In main
      n1 is %d\n",n1);
   return 0 ;
}
```

```
void AddOne(
        int *Pn1){
   *Pn1=*Pn1+1;
   printf ("In AddOne
     n1 is %d\n",*Pn1);
   return;
}
```

5	&n1
n1	**Pn1**

&n1
Pn1

Local copy of Ptr variables

OUTPUT

In AddOne n1 = 6

In main n1 = 6

Passing by reference

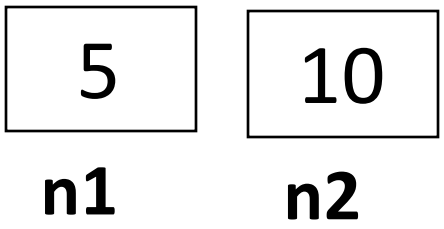
```
#include <stdio.h>
void swap(int *a, int *b)
{
   int temp;
   temp = *a;
   *a = *b;
   *b = temp;
}
void main(){
   int i, j;
   printf("Input two integers: ");
   scanf("%d %d", &i, &j);
   printf("Before swapping: %d %d\n", i, j);
   swap( &i, &j );
   printf("After swapping: %d %d\n", i, j);
}
```

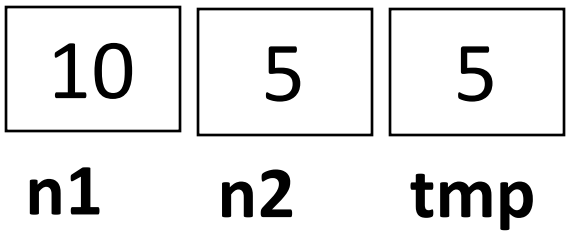
```
Input two integers: 3 5
Before swapping: 3 5
After swapping: 5 3
```

Local Variables and Same Name

```
int main(){
   int n1=5, n2=10;
   swap(n1,n2);
   printf ("In main n1=
      %d n2=%d\n",n1,n2);
   return 0 ;
}
```

```
void swap(int n1,
     int n2){
int tmp;
tmp=n1; n1=n2; n2=tmp;
printf ("In swap n1=
    %d n2=%d\n",n1,n2);
}
```





OUTPUT

In swap n1 = 10 n2 =5

In main n1 = 5 n2 =10

Local copy of variables

Passing by reference

```
int main(){
  int n1=5, n2=10;
printf ("Before swap
n1= %d n2=%d\n",n1,n2);
swap(&n1,&n2);
printf ("After swap n1=
%d n2=%d\n",n1,n2);
return 0 ;}
```

```
void swap(int *Pn1,
     int *Pn2){
int tmp;
tmp=*Pn1;
*Pn1=*Pn2;*Pn2=tmp;
}
```

&n1	&n2	5
Pn1	**Pn2**	**tmp**

5	10
n1	**n2**
***Pn1**	***Pn2**

OUTPUT

Before swap n1 = 5 n2 =10

After swap n1 = 10 n2 =5

Passing Array to Function

```
 float average(float age[]){
    int i; float avg, sum = 0.0;
    for (i = 0; i < 6; ++i) {
        sum = sum +age[i];  age[i]=1;
    }
    avg = (sum / 6);  return avg;
}
```

```
int main(){
 float avg, age[]={23.4,55,22.6,3,40.5,18};
 int i;
 avg = average(age);
 printf("Average age=%f\n", avg);
for(i=0;i<6;++i)printf("%f\n",age[i]);
return 0 ;
}
```

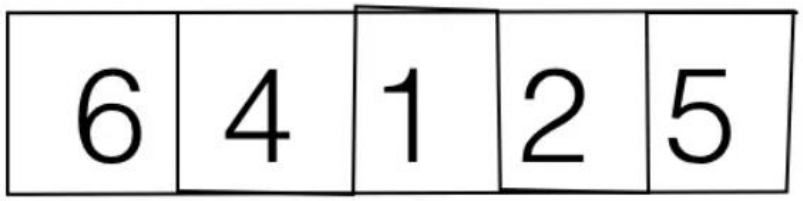
Passing Array to Function

```
//( (float *age) (float age[6]) same
 float average(float age[]){
    int i; float avg, sum = 0.0;
    for (i = 0; i < 6; ++i) {
        sum = sum +age[i];  age[i]=1;
    }
    avg = (sum / 6);  return avg;
}
```

```
int main(){
 float avg, age[]={23.4,55,22.6,3,40.5,18};
 int i;
 avg = average(age);
 printf("Average age=%f\n", avg); // 27.08
 for(i=0;i<6;++i)printf("%f\n",age[i]); //1.000
 return 0 ;
}
```

[age is updated]

Bubble Sorting Algorithm



Iteration 1:	Iteration 2:	Iteration 3:	Iteration 4:
6 4 1 2 5	**4 1** 2 5 6	**1 2** 4 5 6	**1 2** 4 5 6
4 **6 1** 2 5	1 **4 2** 5 6	1 **2 4** 5 6	1 2 4 5 6
4 1 **6 2** 5	1 2 **4 5** 6	1 2 4 5 6	
4 1 2 **6 5**	1 2 4 5 6		
4 1 2 5 6			

Bubble Sorting Algorithm

```
 void swap(int* arr, int i, int j){
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
```

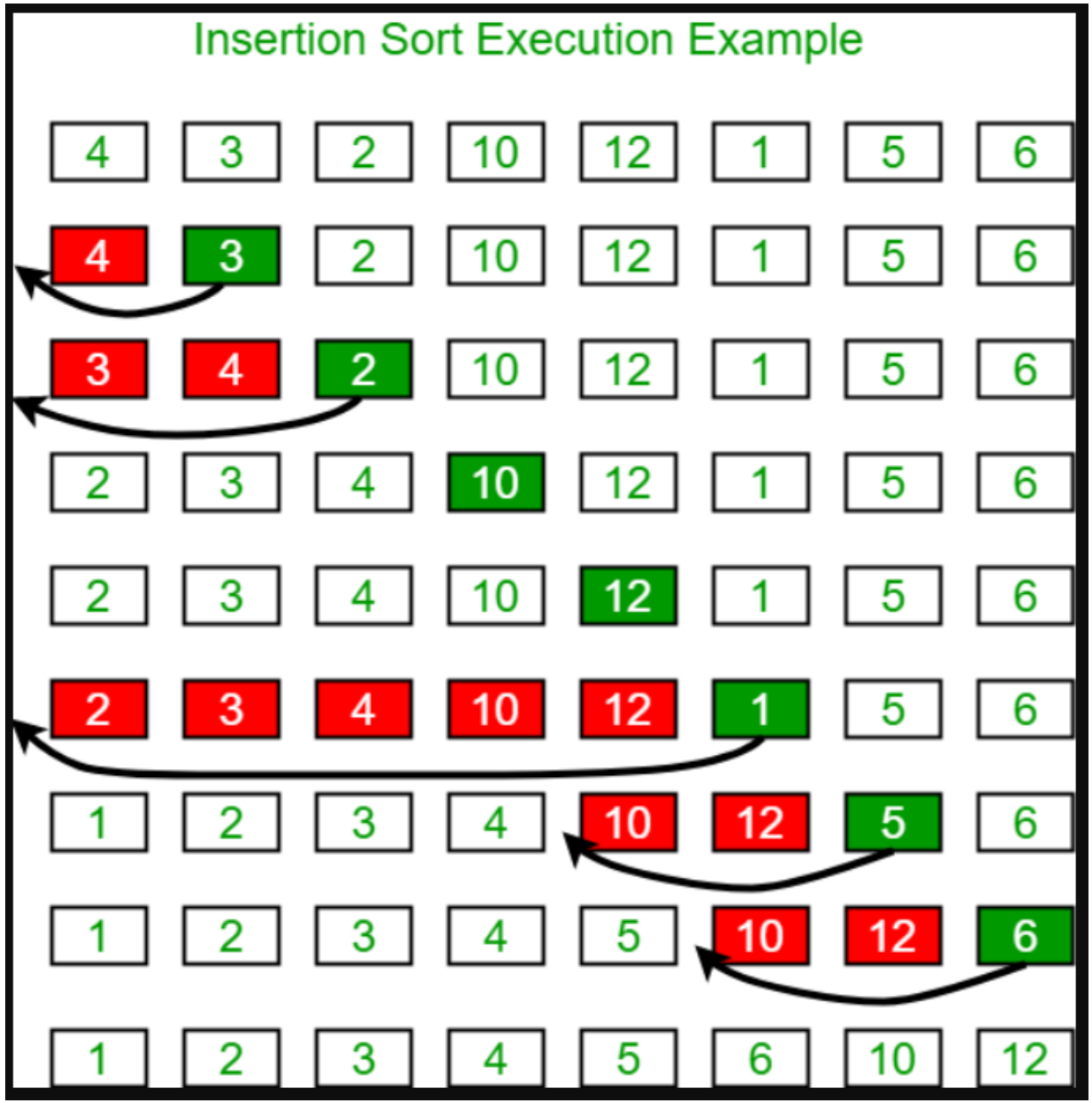
```
void bubbleSort(int arr[], int n){
    int i, j;
    for (i = 0; i < n - 1; i++){
   // Last i elements are already in place
        for (j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1])
                swap(arr, j, j + 1);
     } // end outer for
} // end bubble sort
```

Bubble Sorting Algorithm

```
void printArray(int arr[], int size){
    int i;
    for (i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
```

```
int main(){
    int arr[] = { 5, 1, 4, 2, 8 };
    int N=5;
    printf("Initial array: ");
    printArray(arr, N);
    bubbleSort(arr, N);
    printf("Sorted array: ");
    printArray(arr, N);
    return 0;
}
```

Insertion Sorting Algorithm



Insertion Sorting Algorithm

```
void insertionSort(int arr[], int n){
    int i, key, j;
    for (i = 1; i < n; i++)     {
        key = arr[i];
        j = i - 1;
   /* Move elements of arr[0..i-1], that are
      greater than key, to one position ahead
      of their current position */
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j = j - 1;
        } // end while
        arr[j + 1] = key;
    } // end for
} // end insertionSort
```

Insertion Sorting Algorithm

```
void printArray(int arr[], int size){
    int i;
    for (i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
```

```
int main(){
    int arr[] = { 5, 1, 4, 2, 8 };
    int N=5;
    printf("Initial array: ");
    printArray(arr, N);
    insertionSort(arr, N);
    printf("Sorted array: ");
    printArray(arr, N);
    return 0;
}
```

Thanks